CREDIT CARD APPROVAL PREDICTION SYSTEM

Project Description:

The Credit Card Approval Prediction System is a Machine Learning-based web application developed to automate the process of predicting whether a customer's credit card application is likely to be approved or rejected. The application analyzes applicant information such as demographic details, income, employment, education, family status, housing type, work experience, and credit history to generate a prediction.
The project integrates data preprocessing, feature engineering, machine learning model training, and a Flask-based web application into a complete prediction system. Multiple classification algorithms, including Logistic Regression, Decision Tree, Random Forest, and XGBoost, are trained and evaluated. The best-performing model is selected and deployed using Flask, allowing users to enter applicant details through a web interface and receive instant predictions.
The application is further evaluated using Apache JMeter to verify its performance under multiple user requests, while Git and GitHub are used for version control and project management.

Scenarios

Scenario 1 – Bank Employee Screening Credit Card Applications

A bank employee receives multiple credit card applications every day. Instead of manually reviewing every application, the employee enters the applicant's information into the Credit Card Approval Prediction System. The machine learning model analyzes the applicant's details and instantly predicts whether the application is likely to be approved or rejected, helping reduce processing time.

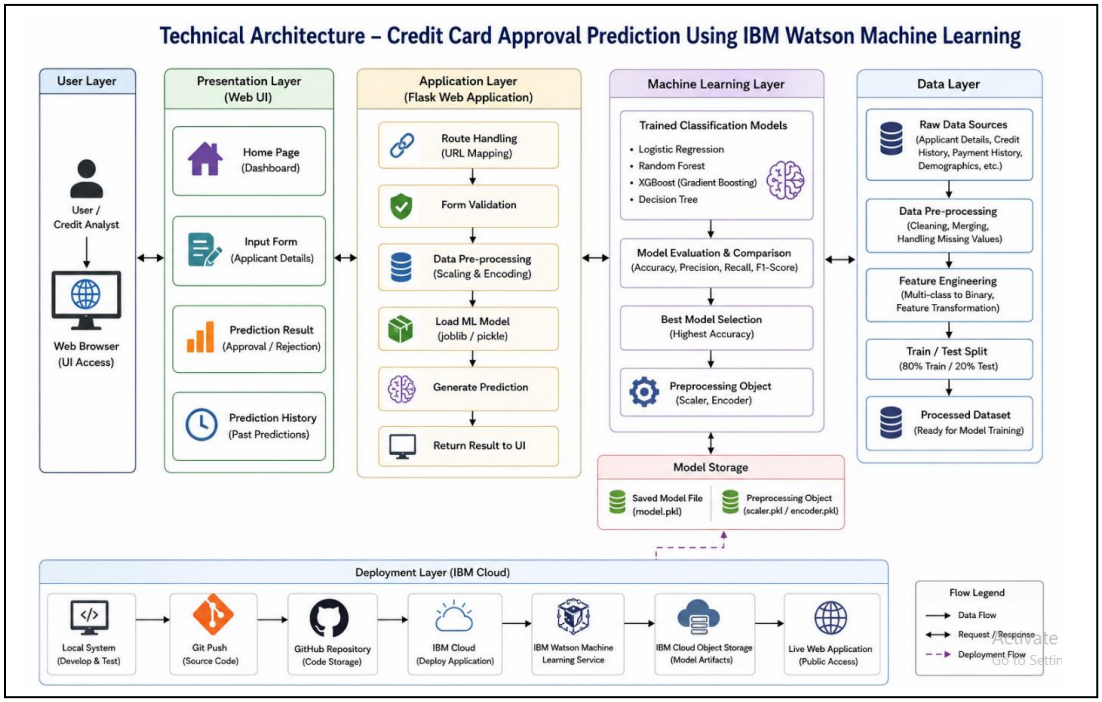
Scenario 2 – Financial Institution Risk Assessment

A financial institution uses the system to perform an initial assessment of applicants before final verification. The prediction provides an additional decision-support tool, allowing officers to prioritize applications and reduce manual effort.

Scenario 3 – Real-Time Credit Card Approval Prediction

A customer service representative enters an applicant's information through the Flask web application. After clicking the **Predict** button, the system preprocesses the input data, applies the trained machine learning model, and displays either **Credit Card Approved** or **Credit Card Rejected** within seconds.

Technical Architecture:



Description:

The Credit Card Approval Prediction System follows a modular machine learning architecture. The frontend is developed using HTML and CSS, while the backend uses the Flask framework. The application collects applicant information through a web interface, performs data preprocessing (label encoding and feature scaling), and sends the processed data to the trained Machine Learning model (best_model.pkl). The model predicts whether the credit card application is Approved or Rejected, and the result is displayed instantly to the user.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

- Python Programming Knowledge: Python Documentation
- Machine Learning Fundamentals: Scikit-learn Documentation
- Flask Framework Knowledge: Flask Documentation
- HTML and CSS Basics: W3Schools HTML & CSS Tutorials
- Data Analysis using Pandas and NumPy: Pandas Documentation, NumPy Documentation
- Model Serialization using Joblib: Joblib Documentation
- Version Control with Git & GitHub: Git Documentation
- Development Environment Setup: Visual Studio Code and Python Installation Guide
- Apache JMeter for Performance Testing: Apache JMeter Documentation
- Jupyter Notebook for Data Analysis and Model Training: Jupyter Documentation

Project Workflow

Milestone 1: Data Collection and Model Development

- **Activity 1.1:** Collect the **Application_Data.csv** dataset and understand the applicant attributes used for credit card approval prediction.
- **Activity 1.2:** Perform data preprocessing by handling missing values, removing duplicates, encoding categorical variables, and scaling numerical features.
- **Activity 1.3:** Train and evaluate multiple machine learning models, including **Logistic Regression, Decision Tree, Random Forest, and XGBoost**.
- **Activity 1.4:** Compare the models based on performance metrics and save the best-performing model (`best_model.pkl`), scaler, and label encoders using Joblib.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core prediction functionality to determine whether a credit card application is **Approved** or **Rejected**.
- **Activity 2.2:** Implement the Flask backend to process user input, preprocess the data, load the trained model, and generate prediction results.

Milestone 3: App.py Development

- **Activity 3.1:** Develop the main application logic in **app.py**, including routing, loading the trained model, processing user input, performing prediction, and displaying the result.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using **HTML** and **CSS** for entering applicant information.
- **Activity 4.2:** Create dynamic Flask templates (`index.html` and `result.html`) using **render_template()** to display the prediction results.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the application locally to ensure accurate predictions and proper functionality.
- **Activity 5.2:** Perform **performance testing using Apache JMeter** by analyzing response time, throughput, and error rate.

Milestone 6: Conclusion

- **Activity 6.1:** Successfully developed a machine learning-based Credit Card Approval Prediction System with a Flask web application.
- **Activity 6.2:** Validated the system through testing and demonstrated its ability to provide fast and reliable approval predictions.

Milestone 1: Data Collection and Model Development

In this milestone, the **Application_Data.csv** dataset is collected and analyzed. Data preprocessing techniques are applied to prepare the dataset for machine learning. Multiple classification algorithms are trained and evaluated, and the best-performing model is selected and saved for deployment in the Flask web application.

Activity 1.1: Collect and Understand the Dataset

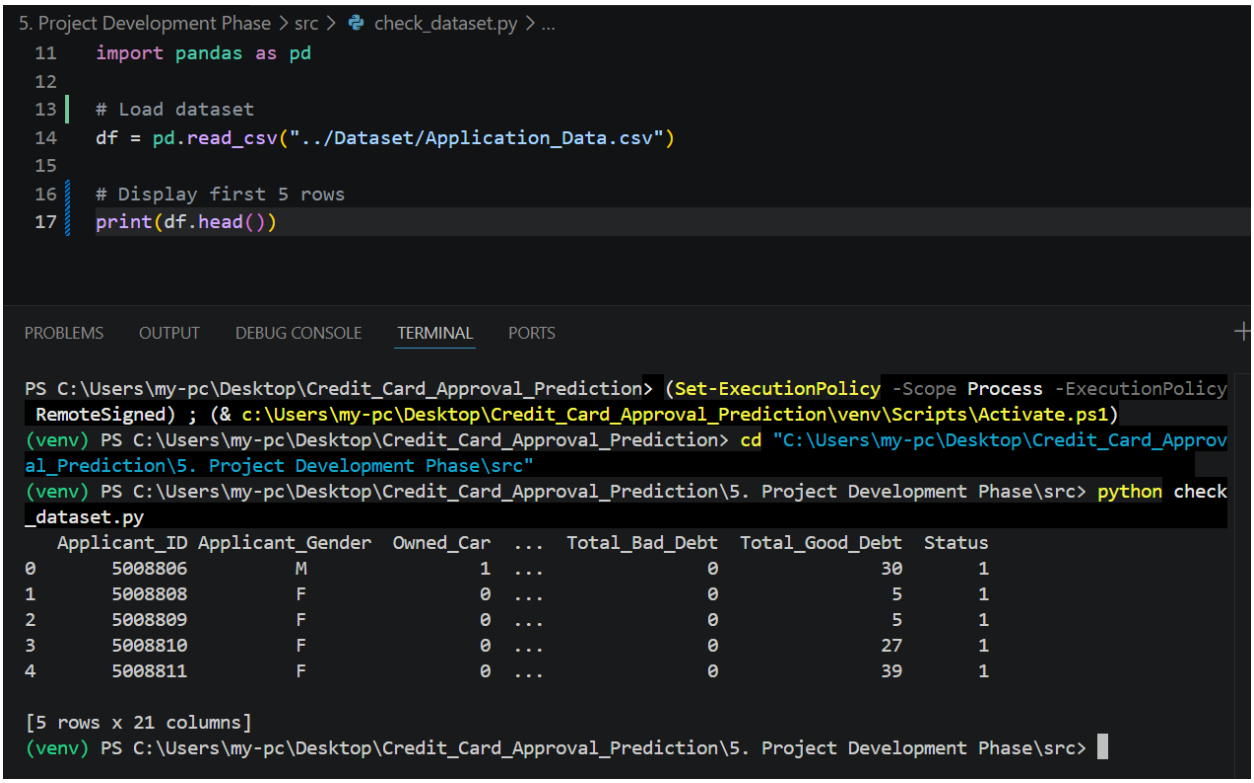
The dataset used in this project is **Application_Data.csv**, which contains applicant details required for credit card approval prediction.

Steps:

- Download or collect the dataset.
- Store it in the **Dataset** folder.
- Load the dataset using Pandas.
- Analyze the dataset structure.
- Identify input features and target variable.

Application_Data.csv

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
1	Applicant_	Applicant_	Owned_Ca	Owned_Re	Total_Chil	Total_Incc	Income_Ty	Education_	Family_Sta	Housing_T	Owned_M	Owned_W	Owned_Ph	Owned_Er	Job_Title	Total_Fam	Applicant_	Years_of_'	Total_Bad	Total_Goo	Status	
2	5008806	M	1	1	0	112500	Working	Secondary	Married	House / ap	1	0	0	0	Security st	2	59	4	0	30	1	
3	5008808	F	0	1	0	270000	Commerci	Secondary	Single / no	House / ap	1	0	1	1	Sales staff	1	53	9	0	5	1	
4	5008809	F	0	1	0	270000	Commerci	Secondary	Single / no	House / ap	1	0	1	1	Sales staff	1	53	9	0	5	1	
5	5008810	F	0	1	0	270000	Commerci	Secondary	Single / no	House / ap	1	0	1	1	Sales staff	1	53	9	0	27	1	
6	5008811	F	0	1	0	270000	Commerci	Secondary	Single / no	House / ap	1	0	1	1	Sales staff	1	53	9	0	39	1	
7	5008815	M	1	1	0	270000	Working	Higher edu	Married	House / ap	1	1	1	1	Accountan	2	47	3	0	6	1	
8	5008819	M	1	1	0	135000	Commerci	Secondary	Married	House / ap	1	0	0	0	Laborers	2	49	4	0	8	1	
9	5008820	M	1	1	0	135000	Commerci	Secondary	Married	House / ap	1	0	0	0	Laborers	2	49	4	0	9	1	
10	5008821	M	1	1	0	135000	Commerci	Secondary	Married	House / ap	1	0	0	0	Laborers	2	49	4	0	9	1	
11	5008822	M	1	1	0	135000	Commerci	Secondary	Married	House / ap	1	0	0	0	Laborers	2	49	4	0	9	1	
12	5008823	M	1	1	0	135000	Commerci	Secondary	Married	House / ap	1	0	0	0	Laborers	2	49	4	0	5	1	
13	5008824	M	1	1	0	135000	Commerci	Secondary	Married	House / ap	1	0	0	0	Laborers	2	49	4	0	4	1	
14	5008825	F	1	0	0	130500	Working	Incomplet	Married	House / ap	1	0	0	0	Accountan	2	30	4	1	25	1	
15	5008826	F	1	0	0	130500	Working	Incomplet	Married	House / ap	1	0	0	0	Accountan	2	30	4	7	23	1	
16	5008830	F	0	1	0	157500	Working	Secondary	Married	House / ap	1	0	1	0	Laborers	2	28	5	2	30	1	
17	5008831	F	0	1	0	157500	Working	Secondary	Married	House / ap	1	0	1	0	Laborers	2	28	5	2	18	1	
18	5008832	F	0	1	0	157500	Working	Secondary	Married	House / ap	1	0	1	0	Laborers	2	28	5	2	33	1	
19	5008836	M	1	1	3	270000	Working	Secondary	Married	House / ap	1	0	0	0	Laborers	5	35	4	0	17	1	
20	5008837	M	1	1	3	270000	Working	Secondary	Married	House / ap	1	0	0	0	Laborers	5	35	4	0	17	1	
21	5008838	M	0	1	1	405000	Commerci	Higher edu	Married	House / ap	1	0	0	0	Managers	3	33	6	0	31	1	
22	5008839	M	0	1	1	405000	Commerci	Higher edu	Married	House / ap	1	0	0	0	Managers	3	33	6	0	14	1	
23	5008840	M	0	1	1	405000	Commerci	Higher edu	Married	House / ap	1	0	0	0	Managers	3	33	6	0	56	1	
24	5008841	M	0	1	1	405000	Commerci	Higher edu	Married	House / ap	1	0	0	0	Managers	3	33	6	0	5	1	
25	5008842	M	0	1	1	405000	Commerci	Higher edu	Married	House / ap	1	0	0	0	Managers	3	33	6	0	9	1	
26	5008843	M	0	1	1	405000	Commerci	Higher edu	Married	House / ap	1	0	0	0	Managers	3	33	6	0	30	1	
27	5008844	M	1	1	0	112500	Commerci	Secondary	Married	House / ap	1	0	1	0	Drivers	2	57	13	0	33	1	
28	5008846	M	1	1	0	112500	Commerci	Secondary	Married	House / ap	1	0	1	0	Drivers	2	57	13	0	12	1	
29	5008847	M	1	1	0	112500	Commerci	Secondary	Married	House / ap	1	0	1	0	Drivers	2	57	13	0	33	1	
30	5008849	M	1	1	0	112500	Commerci	Secondary	Married	House / ap	1	0	1	0	Drivers	2	57	13	0	12	1	
31	5008850	M	1	1	0	112500	Commerci	Secondary	Married	House / ap	1	0	1	0	Drivers	2	57	13	0	26	1	



Activity 1.2: Research and Select the Appropriate Machine Learning Model :

The following activities were carried out to identify the most suitable machine learning model for the Credit Card Approval Prediction system:

- **Understand the Project Requirements:** Analyze the objective of predicting whether a credit card application should be **Approved** or **Rejected** based on applicant information such as income, age, employment, education, family status, and credit history.
- **Study Different Classification Algorithms:** Research various machine learning classification algorithms including **Logistic Regression**, **Decision Tree**, **Random Forest**, and **XGBoost** to understand their working principles, advantages, and limitations.
- **Preprocess the Dataset:** Prepare the dataset by handling missing values, removing duplicate records, encoding categorical features, and scaling numerical features to improve model performance.
- **Train and Evaluate Multiple Models:** Train each classification algorithm using the processed dataset and evaluate their performance using metrics such as **Accuracy**, **Precision**, **Recall**, **F1-Score**, and **Confusion Matrix**.
- **Select the Best Performing Model:** Compare the evaluation results of all trained models and choose the model with the highest prediction accuracy and overall performance. Save the selected model along with the scaler and label encoders using **Joblib** for deployment in the Flask application.
- **Selected Model: Random Forest Classifier** *(replace this with **XGBoost** or another model if that is the one you actually deployed as* `best_model.pkl`*.)*

Activity 1.3: Define the Architecture of the Application

The following activities were performed to design the architecture of the **Credit Card Approval Prediction** system:

- **Design the System Architecture:** Create a high-level architecture illustrating the interaction between the user interface, Flask web application, machine learning model, and dataset.
- **Define the User Interface:** Develop a user-friendly web interface using **HTML** and **CSS** that allows users to enter applicant details such as age, income, education, occupation, family status, and other required information.
- **Design the Flask Backend:** Implement the backend using the **Flask** framework to receive user input, validate the submitted data, preprocess the features, and communicate with the trained machine learning model.
- **Integrate the Machine Learning Model:** Load the trained classification model (`best_model.pkl`), scaler (`scaler.pkl`), and label encoders (`label_encoders.pkl`) using **Joblib**. Transform the input data into the required format before making predictions.
- **Implement the Prediction Workflow:** Process the applicant information by performing categorical encoding and feature scaling, pass the processed data to the trained model, and generate the prediction result as **Credit Card Approved** or **Credit Card Rejected**.
- **Display Prediction Results:** Present the prediction outcome on the result page with a clear message indicating whether the credit card application is approved or rejected.

  User

  ↓

  HTML Form

  ↓

  Flask Backend

  ↓

  Preprocessing

  ↓

  ML Model

  ↓

  Prediction

  ↓

  Result Page

Activity 1.4: Set Up the Development Environment

The following activities were performed to prepare the development environment for the **Credit Card Approval Prediction** system.

- **Install Python and Required Tools:** Install Python 3.x, Visual Studio Code, Git, and GitHub Desktop to support project development and version control.
- **Create a Virtual Environment:** Create and activate a virtual environment to isolate project dependencies :

  python -m venv venv

  Windows:

  venv\Scripts\activate

- **Install Required Libraries:** Install all required Python packages listed in the `requirements.txt` file.

  pip install -r requirements.txt

The major libraries used in this project include:

- Flask
- Pandas
- NumPy
- Scikit-learn
- Joblib
- XGBoost (if used)
- **Organize the Project Structure:** Create the project folders for datasets, source code, model files, Flask application, documentation, testing, and deployment according to the project template.

Example structure:

Credit_Card_Approval_Prediction/

│

├── Dataset/

├── model/

├── notebooks/

├── src/

├── flask_app/

│ ├── templates/

```
│   └── static/
├── deployment/
├── requirements.txt
└── README.md
```

- **Load the Dataset:** Place the `Application_Data.csv` dataset in the Dataset folder and verify that it can be loaded successfully using Pandas for preprocessing and model training.
- **Prepare the Flask Application:** Configure the Flask application (`app.py`) to load the trained model (`best_model.pkl`), scaler (`scaler.pkl`), and label encoders (`label_encoders.pkl`) using Joblib.

- **Verify the Development Environment:** Run the Flask application locally and ensure that the home page loads successfully and the prediction functionality works correctly.

```
python app.py
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Credit Card Approval Prediction Features

The core functionality of the **Credit Card Approval Prediction** system was developed to predict whether a credit card application should be **Approved** or **Rejected** using machine learning techniques.

The following activities were carried out:

- **Develop the Applicant Information Form:** Create a user-friendly input form using **HTML** and **CSS** to collect applicant details such as Applicant ID, Gender, Income, Education, Employment, Family Status, Housing Type, Age, Years of Working, Good Debt, Bad Debt, and other required information.
- **Validate User Input:** Implement input validation to ensure all mandatory fields are completed and that numerical values such as income, age, and family members are valid before processing.
- **Preprocess Applicant Data:** Convert the user input into a structured format using **Pandas DataFrame**. Encode categorical features using the saved **Label Encoders** and normalize numerical features using the trained **Scaler**.
- **Generate Credit Card Approval Prediction:** Pass the processed data to the trained machine learning model (`best_model.pkl`) to predict whether the applicant is eligible for credit card approval.
- **Display the Prediction Result:** Present the prediction result to the user through the Flask application with one of the following messages:
  - **Credit Card Approved** ✅
  - **Credit Card Rejected** ❌
- **Handle Invalid Inputs and Errors:** Implement exception handling to manage missing values, invalid inputs, model loading issues, and unexpected system errors to improve the reliability of the application.

Credit Card Approval Prediction

Applicant ID

Gender
Male

Owned Car
Yes

Owned Realty
Yes

Total Children

Total Income

Income Type
Working

Education
Higher education

Family Status
Married

Housing Type
House / apartment

Mobile Phone
Yes

Work Phone
Yes

Phone
Yes

Email
Yes

Job Title
Managers

Total Family Members

Applicant Age

Years of Working

Total Bad Debt

Total Good Debt

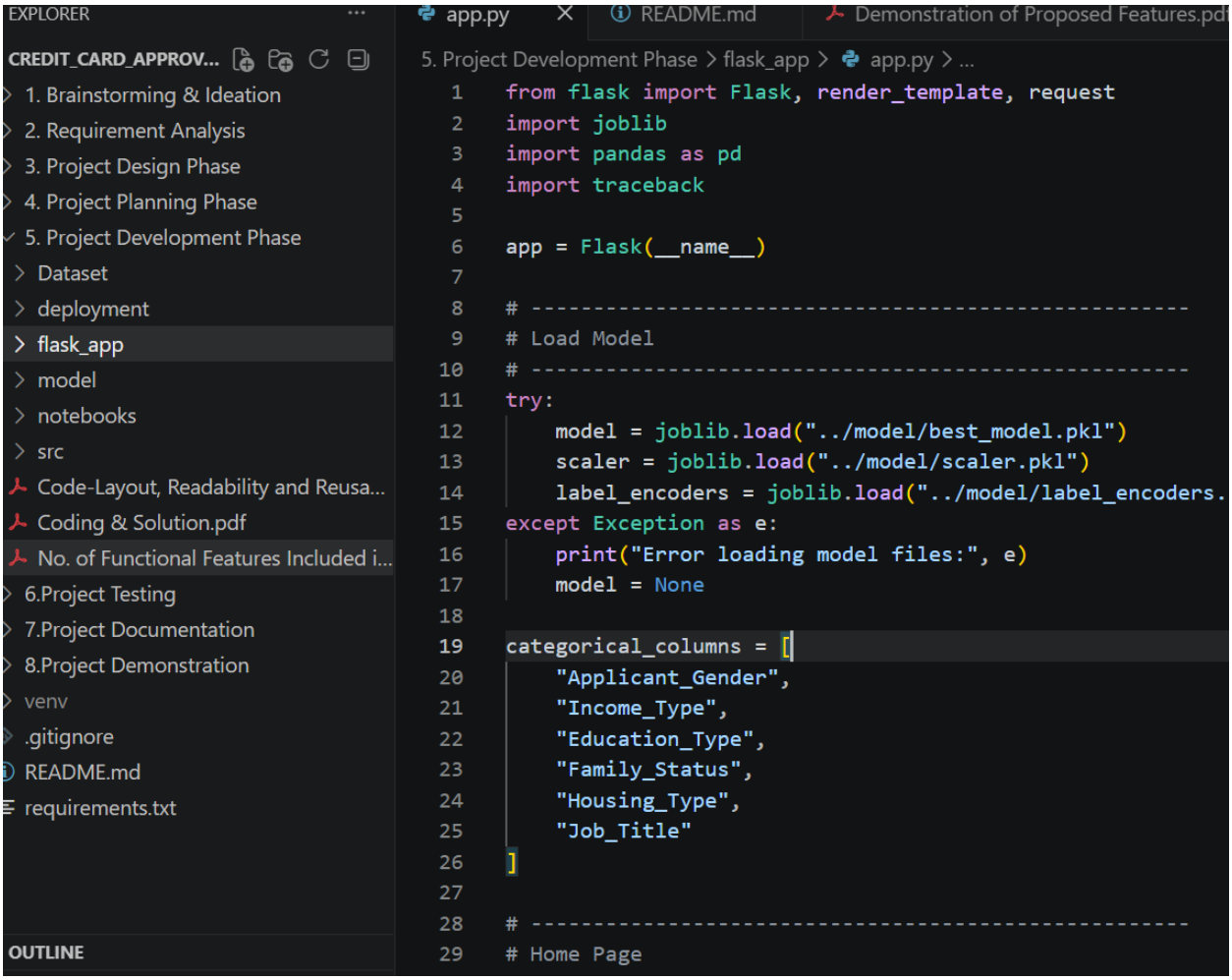
Predict Credit Card Approval

Activity 2.2: Implement the Flask Backend

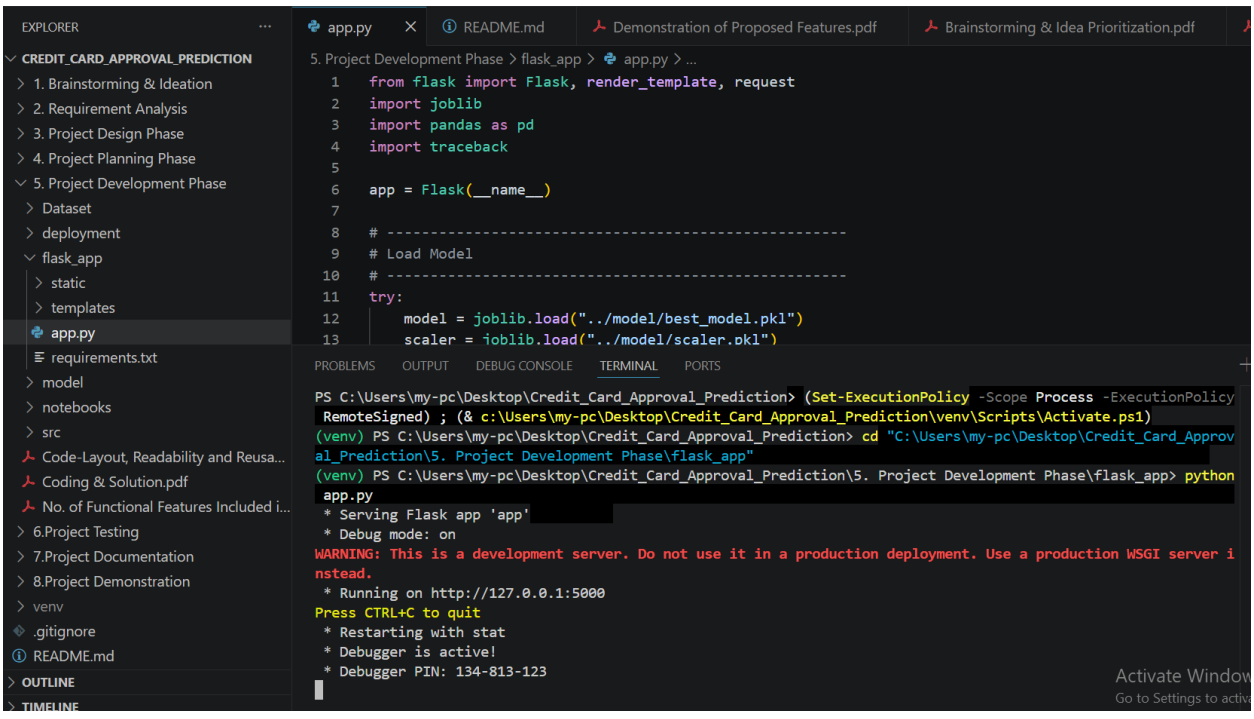
The Flask backend was developed to handle user requests, preprocess applicant information, interact with the trained machine learning model, and display the prediction results.

The following activities were performed:

- **Define Flask Routes:** Create routes in `app.py` for the home page (`/`) and prediction page (`/predict`). The home route displays the applicant input form, while the prediction route processes the submitted data.
- **Collect User Input:** Retrieve applicant details submitted through the HTML form using Flask's `request.form` object. The application collects information such as Applicant ID, Gender, Income, Education, Employment Type, Family Status, Housing Type, Age, Years of Working, Good Debt, and Bad Debt.
- **Validate Input Data:** Verify that all required fields are provided and validate numerical inputs such as income, age, total family members, and debt values before processing.
- **Preprocess Applicant Data:** Convert the collected input into a Pandas DataFrame. Encode categorical features using the saved **Label Encoders** and normalize numerical features using the trained **Scaler** to match the format used during model training.
- **Generate Prediction:** Load the trained machine learning model (`best_model.pkl`) using **Joblib** and generate the prediction based on the processed applicant information.
- **Display Prediction Result:** Return the prediction to the user by rendering the `result.html` page with one of the following outcomes:
    - **Credit Card Approved** ✅
    - **Credit Card Rejected** ❌
- **Implement Error Handling:** Handle invalid inputs, missing form fields, model loading errors, and unexpected exceptions to ensure the application remains stable and provides meaningful error messages to users.


EXPLORER
CREDIT_CARD_APPROV...
1. Brainstorming & Ideation
2. Requirement Analysis
3. Project Design Phase
4. Project Planning Phase
5. Project Development Phase
Dataset
deployment
flask_app
model
notebooks
src
Code-Layout, Readability and Reusa...
Coding & Solution.pdf
No. of Functional Features Included i...
6.Project Testing
7.Project Documentation
8.Project Demonstration
venv
.gitignore
README.md
requirements.txt
OUTLINE
app.py
README.md
Demonstration of Proposed Features.pdf
5. Project Development Phase > flask_app > app.py > ...
from flask import Flask, render_template, request
import joblib
import pandas as pd
import traceback
app = Flask(__name__)
# ----------
# Load Model
# ----------
try:
model = joblib.load("../model/best_model.pkl")
scaler = joblib.load("../model/scaler.pkl")
label_encoders = joblib.load("../model/label_encoders.
except Exception as e:
print("Error loading model files:", e)
model = None
categorical_columns = [
"Applicant_Gender",
"Income_Type",
"Education_Type",
"Family_Status",
"Housing_Type",
"Job_Title"
]

Home Page


EXPLORER
CREDIT_CARD_APPROVAL_PREDICTION
1. Brainstorming & Ideation
2. Requirement Analysis
3. Project Design Phase
4. Project Planning Phase
5. Project Development Phase
Dataset
deployment
flask_app
static
templates
app.py
requirements.txt
model
notebooks
src
Code-Layout, Readability and Reusa...
Coding & Solution.pdf
No. of Functional Features Included i...
6.Project Testing
7.Project Documentation
8.Project Demonstration
venv
.gitignore
README.md
OUTLINE
TIMELINE
app.py
README.md
Demonstration of Proposed Features.pdf
Brainstorming & Idea Prioritization.pdf
5. Project Development Phase > flask_app > app.py > ...
from flask import Flask, render_template, request
import joblib
import pandas as pd
import traceback
app = Flask(__name__)
# ----------
# Load Model
# ----------
try:
model = joblib.load("../model/best_model.pkl")
scaler = joblib.load("../model/scaler.pkl")
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\my-pc\Desktop\Credit_Card_Approval_Prediction> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\my-pc\Desktop\Credit_Card_Approval_Prediction\venv\Scripts\Activate.ps1)
(venv) PS C:\Users\my-pc\Desktop\Credit_Card_Approval_Prediction> cd "C:\Users\my-pc\Desktop\Credit_Card_Approval_Prediction\5. Project Development Phase\flask_app"
(venv) PS C:\Users\my-pc\Desktop\Credit_Card_Approval_Prediction\5. Project Development Phase\flask_app> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 134-813-123
Activate Window
Go to Settings to activ

.

Milestone 3: Model Training and Evaluation

Activity 3.1: Train and Evaluate the Machine Learning Model

The machine learning models were trained and evaluated to identify the most suitable algorithm for predicting credit card approval based on applicant information.

The following activities were performed:

- **Load the Processed Dataset:** Import the preprocessed credit card application dataset and separate the input features from the target variable (Credit Card Approval Status).
- **Split the Dataset:** Divide the dataset into training and testing sets to evaluate the performance of the machine learning models on unseen data.
- **Train Multiple Classification Models:** Train different classification algorithms, including **Logistic Regression**, **Decision Tree**, **Random Forest**, and **XGBoost**, to compare their predictive performance.
- **Evaluate Model Performance:** Measure the performance of each model using evaluation metrics such as **Accuracy**, **Precision**, **Recall**, **F1-Score**, and **Confusion Matrix**.
- **Select the Best Model:** Compare the evaluation results and select the model that achieved the highest accuracy and overall performance for deployment in the application.
- **Save the Trained Model:** Store the selected model (`best_model.pkl`), feature scaler (`scaler.pkl`), and label encoders (`label_encoders.pkl`) using **Joblib** for use in the Flask application.
- **Verify Prediction Accuracy:** Test the saved model with sample applicant records to ensure that it accurately predicts whether a credit card application should be **Approved** or **Rejected**.

Activity 3.2: Integrate the Trained Model with the Flask Application

The trained machine learning model was integrated with the Flask web application to provide real-time credit card approval predictions based on user input.

The following activities were performed:

- **Load the Trained Model:** Load the trained machine learning model (`best_model.pkl`) using the **Joblib** library during application startup to ensure efficient prediction.
- **Load Supporting Files:** Load the **Scaler (`scaler.pkl`)** and **Label Encoders (`label_encoders.pkl`)** to preprocess user input in the same way as the training data.
- **Receive Applicant Information:** Collect applicant details entered through the Flask web interface using HTML forms and the `request.form` method.
- **Preprocess Input Data:** Convert the submitted applicant information into a Pandas DataFrame, encode categorical features, and scale numerical features before passing them to the trained model.
- **Generate Predictions:** Use the integrated machine learning model to predict whether the applicant's credit card request should be **Approved** or **Rejected**.
- **Display the Prediction Result:** Show the prediction outcome on the result page with clear messages such as:
    - **Credit Card Approved** ✅

  - **Credit Card Rejected ✗**
- **Implement Exception Handling:** Include error handling to manage invalid inputs, missing values, model loading failures, and unexpected runtime errors, ensuring the application remains reliable and user-friendly.
- **Verify End-to-End Functionality:** Test the complete application by submitting multiple applicant records and confirming that predictions are generated correctly and displayed successfully.

Milestone 4: Testing

Activity 4.1: Perform Functional Testing

The following activities were carried out to verify that the **Credit Card Approval Prediction** application functions correctly:

- **Test the User Interface:** Verify that the home page, applicant input form, and result page load correctly without any display issues.
- **Validate User Inputs:** Test the application with both valid and invalid applicant information to ensure that appropriate validation messages are displayed.
- **Verify Data Processing:** Confirm that the entered applicant details are correctly converted into the required format, encoded, and scaled before being passed to the machine learning model.
- **Test Prediction Functionality:** Verify that the trained model correctly predicts whether the credit card application is **Approved** or **Rejected**.
- **Verify Error Handling:** Test scenarios such as missing inputs, invalid values, and incorrect data formats to ensure that the application handles exceptions gracefully without crashing.
- **Validate End-to-End Workflow:** Ensure that the complete workflow—from entering applicant details to displaying the prediction result—functions correctly.

Activity 4.2: Perform Performance Testing

The following activities were carried out to evaluate the performance of the **Credit Card Approval Prediction** application:

- **Configure Apache JMeter:** Create a JMeter Test Plan to simulate multiple users accessing the Flask application simultaneously.
- **Generate Concurrent Requests:** Configure Thread Groups in JMeter to send multiple HTTP requests to the application and evaluate its performance under load.
- **Measure Response Time:** Record the average, minimum, and maximum response times for prediction requests.
- **Monitor Throughput:** Measure the number of requests processed successfully per second.
- **Analyze Error Rate:** Verify that the application processes requests without failures or unexpected errors during performance testing.
- **Review Performance Reports:** Analyze JMeter reports and graphs to evaluate the application's stability, response time, and overall performance.
- **Document Testing Results:** Record the performance metrics, screenshots, observations, and conclusions for inclusion in the project documentation.

Expected Outcome

- All application features work correctly.

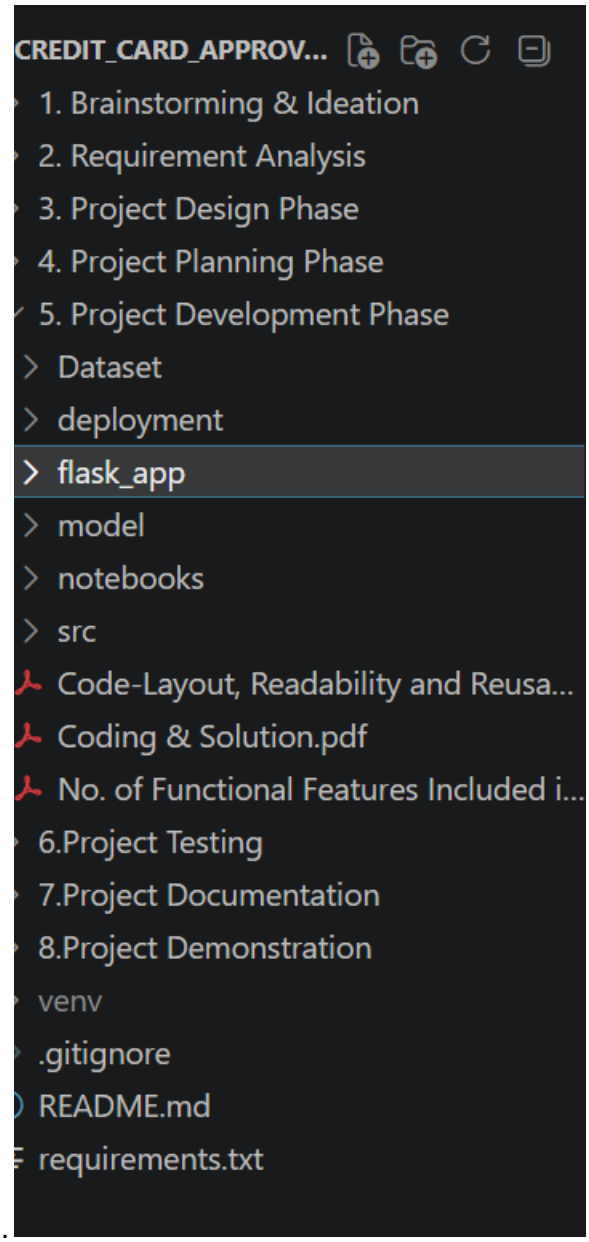
- Input validation functions as expected.
- Credit card approval predictions are generated accurately.
- The Flask application remains stable under multiple user requests.
- Apache JMeter reports indicate acceptable response times and low error rates.
- The application is ready for deployment.

Milestone 5: Deployment

5.1 Prepare the Deployment Environment

The following activities were performed:

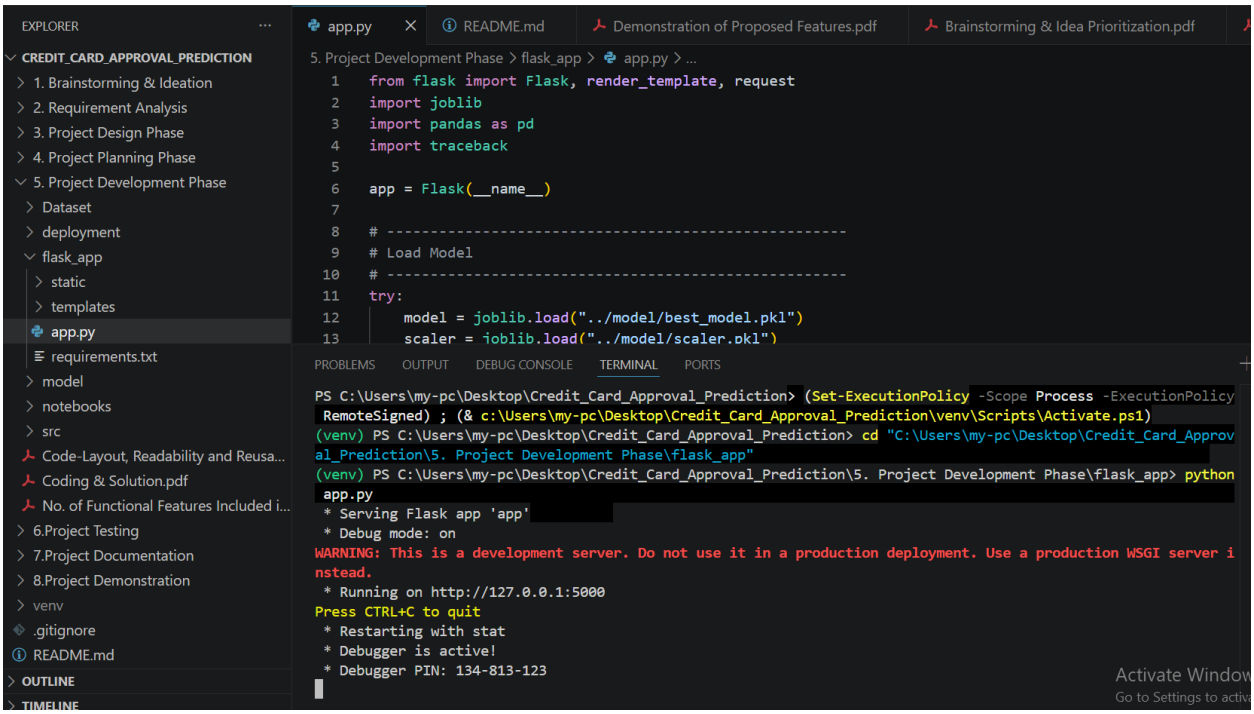
- Install Python and configure the development environment.
- Create and activate a virtual environment for the project.
- Install all required Python libraries using the `requirements.txt` file.
- Verify that all dependencies are installed successfully.
- Organize the project folders, datasets, model files, templates, and static resources.
- Ensure the trained machine learning model, scaler, and label encoder files are available for deployment



5.2 Configure and Deploy the Application

The following activities were performed:

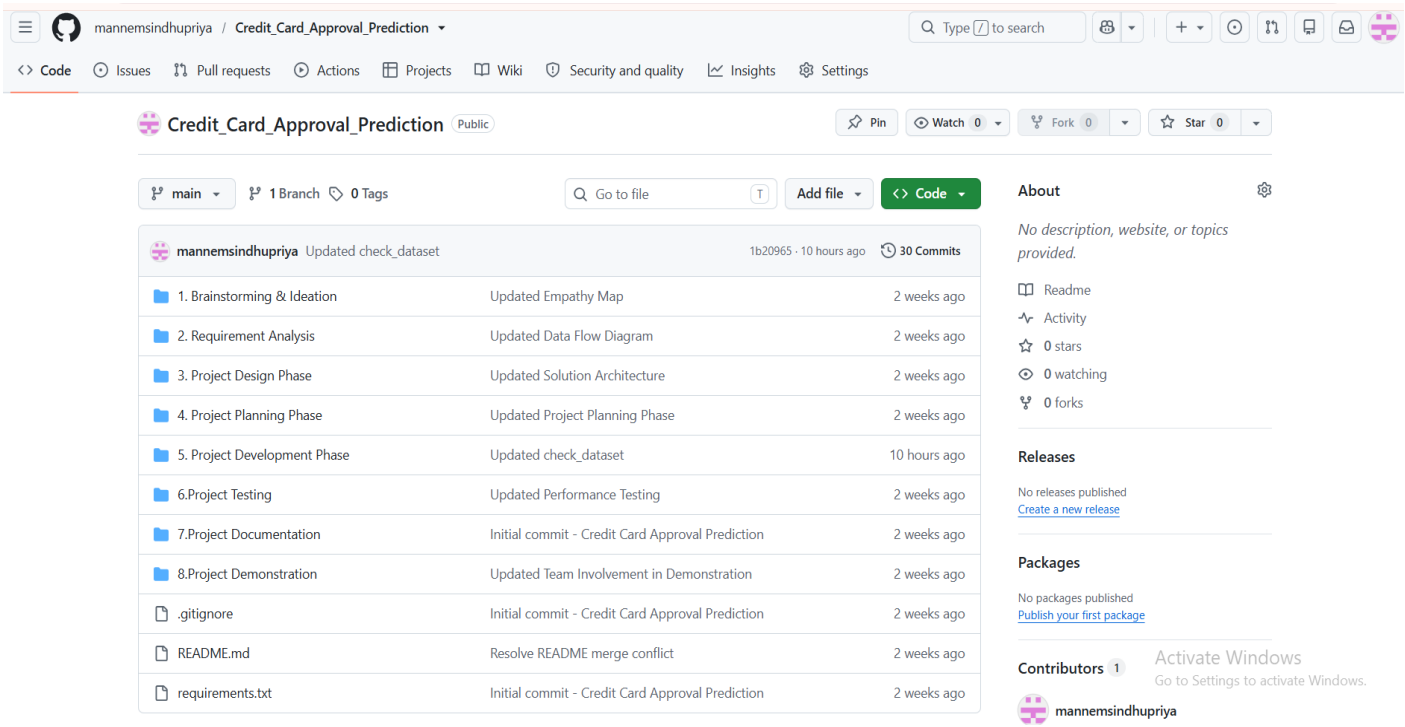
- Configure the Flask application (`app.py`) for deployment.
- Load the trained machine learning model (`best_model.pkl`).
- Load the scaler (`scaler.pkl`) and label encoders (`label_encoders.pkl`).
- Configure the application routes for the home page and prediction functionality.
- Launch the Flask application on the local server.
- Verify that the application starts successfully without any errors.
- Access the application through a web browser and ensure all pages load correctly.



5.3 Validate the Deployed Application

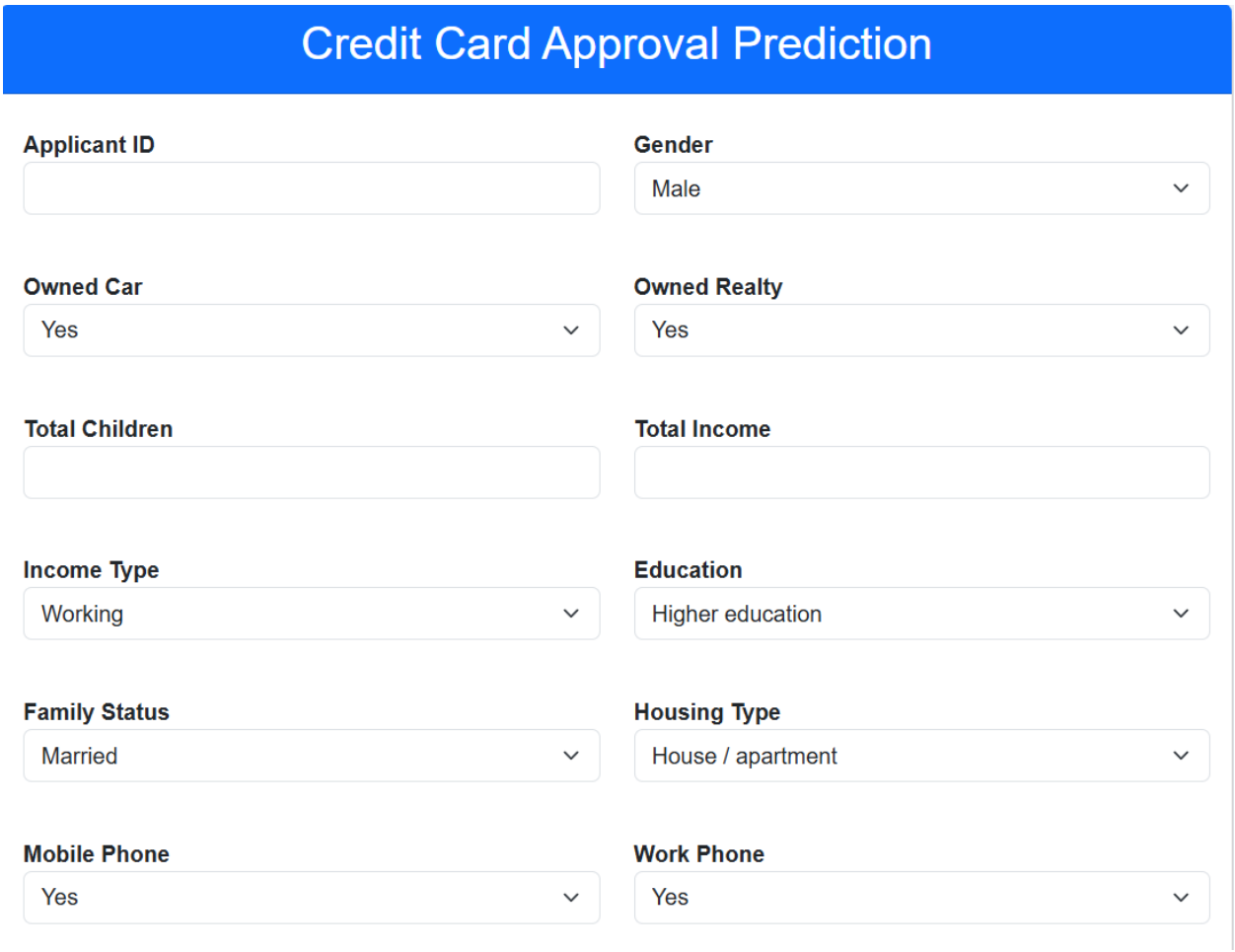
The following activities were performed:

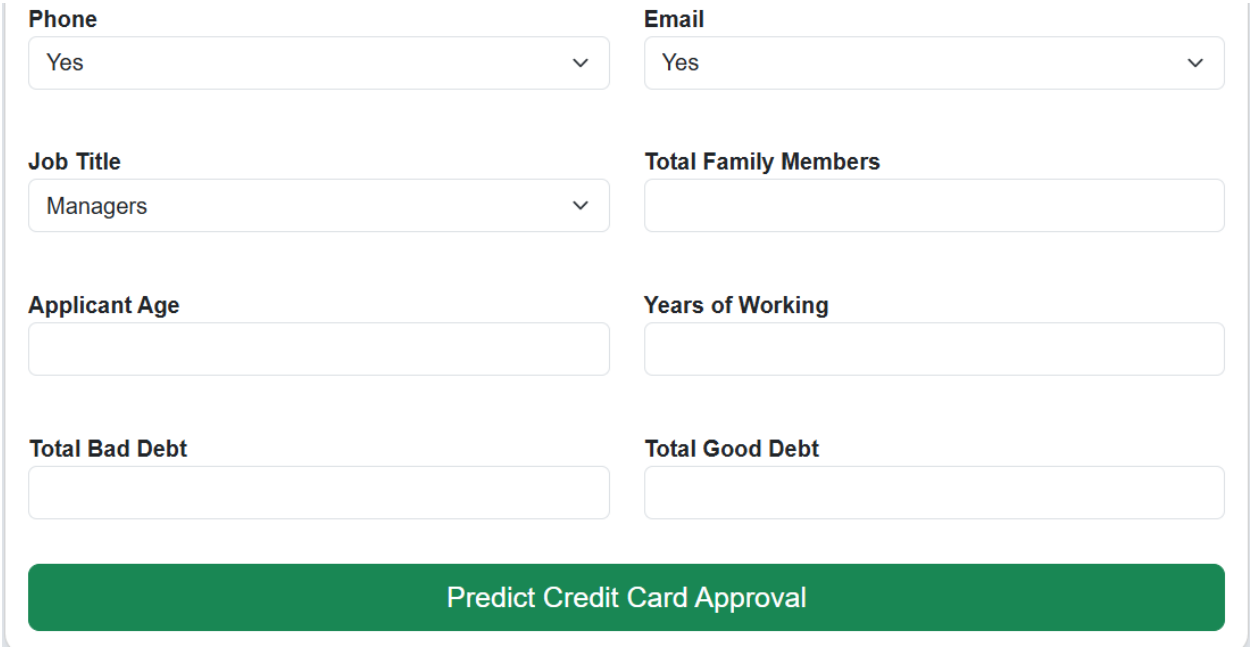
- Test the application using different credit card applicant records.
- Verify that user inputs are processed correctly.
- Confirm that the machine learning model accurately predicts whether the credit card application is **Approved** or **Rejected**.
- Validate the prediction results displayed on the web interface.
- Verify exception handling for invalid or incomplete user inputs.
- Perform end-to-end testing to ensure the frontend, backend, and machine learning model work together successfully.
- Capture deployment screenshots and update the GitHub repository with the final project files.
- Confirm that the application is ready for demonstration and final project submission.



Exploring the Website's Web Pages:

Home / Generator Page:

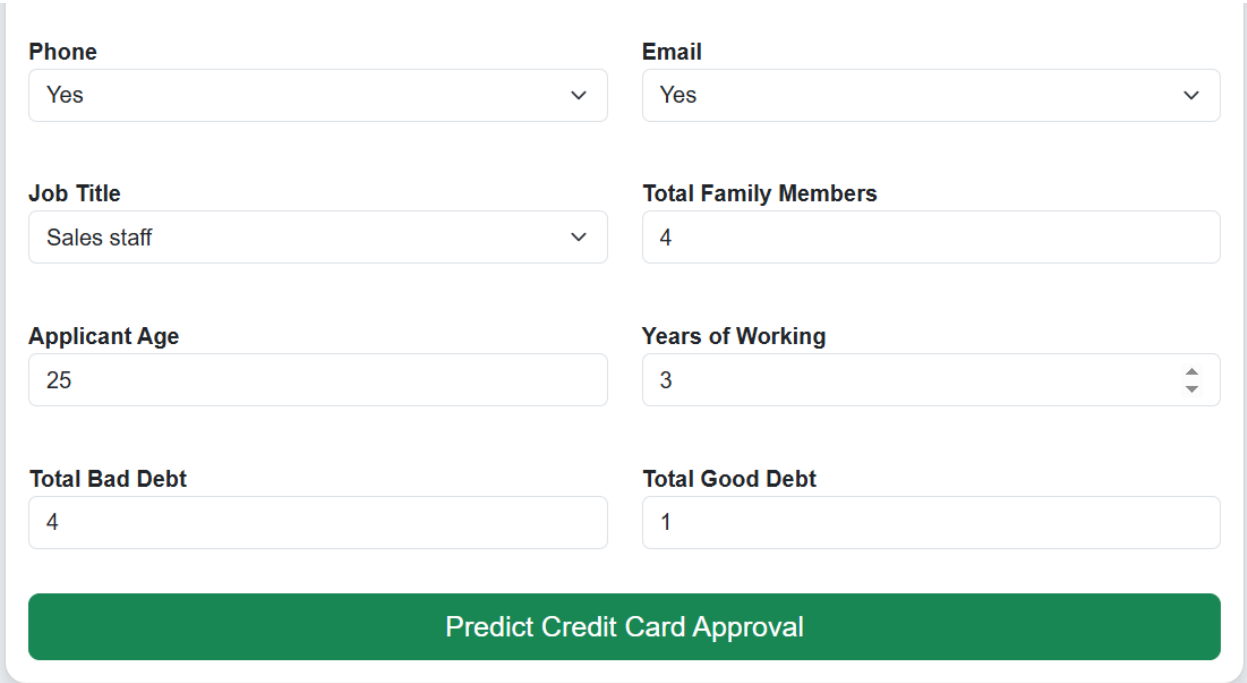




Description:

The Home Page provides the user interface for the Credit Card Approval Prediction System. Users can enter applicant information such as gender, income, education, employment details, family status, and credit history. After filling in all required fields, users click the **Predict** button to check whether the credit card application is likely to be approved or rejected.

Input Section:



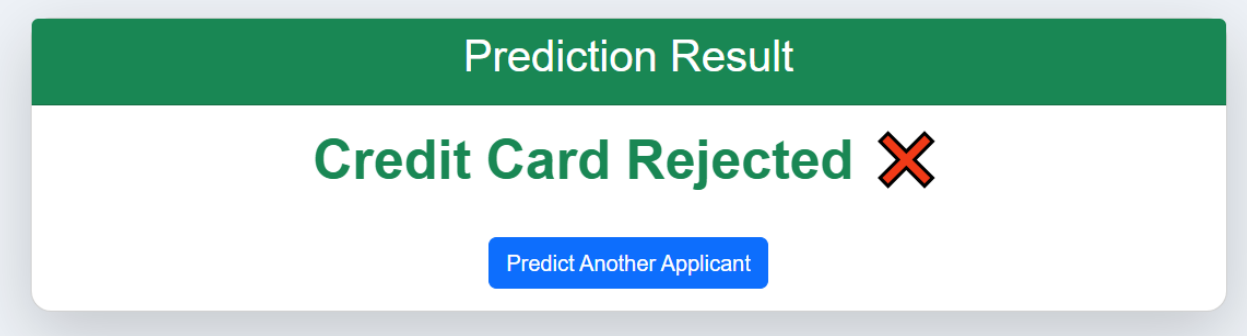
Description:

The Input Section collects all applicant information required for prediction. The form includes fields such as:

- Applicant ID
- Gender
- Car Ownership
- Property Ownership
- Number of Children
- Total Income
- Income Type
- Education Type
- Family Status
- Housing Type
- Mobile Phone Ownership
- Work Phone Ownership
- Personal Phone Ownership
- Email Ownership
- Job Title
- Total Family Members
- Applicant Age
- Years of Working
- Total Bad Debt
- Total Good Debt

After entering the details, the user submits the form by clicking the **Predict** button.

Results Section:



Description:

The Prediction Result Page displays the output generated by the trained machine learning model. Based on the applicant's information, the system predicts whether the credit card application is **Approved** or **Rejected**. The prediction is displayed clearly along with an option to return to the Home Page and perform another prediction.

Milestone 6: Conclusion

The Credit Card Approval Prediction System was successfully designed and developed using Machine Learning and Flask. The application analyzes applicant information such as income, employment, education, family status, and credit history to predict whether a credit card application is likely to be approved or rejected. Multiple machine learning algorithms were trained and evaluated, and the best-performing model was selected to improve prediction accuracy and reliability.

A user-friendly web interface was developed to allow users to enter applicant details and receive instant prediction results. The application also includes input validation and error handling to ensure smooth and reliable operation. Throughout the project, key phases such as data preprocessing, exploratory data analysis, feature engineering, model training, evaluation, testing, and deployment were completed successfully.

Overall, the project demonstrates how Machine Learning can assist financial institutions in making faster, more consistent, and data-driven credit approval decisions. In the future, the system can be enhanced by integrating real-time banking data, advanced fraud detection techniques, explainable AI (XAI), cloud deployment, and stronger security features to make it more suitable for real-world banking applications.